

# The refactor

One bad handler, rebuilt, with tests proving the behaviour survived and six defects did not

`pages/api/subscriptions/change-plan.ts` — about 400 requests a day, every one of them changing what a customer pays. It is finding 6 in the [assessment](#) and six other handlers look like it.

Everything on this page runs: `npm test` in `task_b/` executes 29 tests against both versions.

## Before

TS

```
import { database } from "../support/database";
import { payments } from "../support/payments";
import { mailer } from "../support/mailer";

const PLAN_PRICES: Record<string, number> = {
  starter: 2400,
  classic: 3600,
  deluxe: 5400,
};

const PAYMENT_API_KEY = "sk_live_<the-real-one-was-committed-here-in-2021>";

export async function POST(req: Request) {
  const body = await req.json();
  const userId = req.headers.get("x-user-id");

  const rows = await database.query(
    `SELECT * FROM subscriptions WHERE id = '${body.subscriptionId}'`
  );
  const sub = rows[0];

  let price = PLAN_PRICES[body.newPlan];
  if (sub.discount_percent) {
    price = price - (price * Number(sub.discount_percent)) / 100;
  }
  if (body.promoCode === "SAVE10") {
    price = price * 0.9;
  }
  price = Math.round(price);

  await database.query(
    `UPDATE subscriptions SET plan = '${body.newPlan}', price_cents = ${price}, updated_at =
    '${new Date().toISOString()}' WHERE id = '${body.subscriptionId}'`
  );

  await payments.updateSubscriptionPrice(String(sub.payment_ref), price);

  await database.query(
```

```

    `INSERT INTO subscription_events (subscription_id, type, detail, actor_id, created_at) VALUES
    ('${body.subscriptionId}', 'plan_changed', '${body.newPlan}', '${userId}', '${new
    Date().toISOString()}')`
    );

    await mailer.sendNow(String(sub.email), `Your plan is now ${body.newPlan}`);

    return Response.json(sub);
}

```

It is short, it reads top to bottom, and it does what its name says. That is why it survived six years of review.

## What is wrong with it

- 1. It never checks who is asking.** `userId` is read and then only used to stamp the audit row. Nothing compares it to `sub.user_id`, so any signed-in customer can change any other customer's plan by passing a different `subscriptionId`.
- 2. The id goes straight into SQL.** A value of `nope' OR '1'='1` makes the `WHERE` match every row, so the `UPDATE` rewrites every subscription in the table.
- 3. An unknown plan produces `NaN`.** Nothing validates `newPlan`. `PLAN_PRICES["premium"]` is `undefined`, and `NaN` ends up in the database and at the payment provider.
- 4. Three writes, no transaction, an external call in the middle.** When Stripe fails, the customer is on the new plan, has not been charged, and — because the failure happens before the `INSERT` — nothing records that it happened.
- 5. The email is sent inside the request.** If mail is slow the customer waits; if it throws, the request 500s *after* the money moved, and the client's retry charges again.
- 6. The response is stale and leaks.** `sub` was read before the update, so the caller gets the old plan back. It is the raw row, so it includes `internal_notes` and `payment_ref`.

Plus the live Stripe key in the source, which is fixed by rotation, not by refactoring.

## Step 1: pin the behaviour first

Before changing anything, eight tests describing what the handler does today — including behaviour I think is wrong. The promo code stacking on top of the account discount might well be a bug, but that is a pricing decision, not a refactor decision.

The tests take the handler as a parameter and run against **both** implementations:

TS

```
describe.each(implementations)("%s: a customer upgrades their own plan", (_name, handler) => {
  it("stacks a promo code on top of the account discount", async () => {
    const outcome = await call(handler, {
      actorId: "user_ben",
      body: { subscriptionId: "sub_ben", newPlan: "deluxe", promoCode: "SAVE10" },
    });

    expect(outcome.paymentCalls).toEqual([
      { reference: "pay_ben_4417", priceCents: 3645 }
    ]);
  });
});
```

If the new version charges a different amount, applies the discounts in a different order, writes a different event, or touches another customer's row, the suite goes red.

## Step 2: after

Three files, each with one job.

### pricing.ts — the rules, nothing else

TS

```
export const PLAN_PRICES = { starter: 2400, classic: 3600, deluxe: 5400 } as const;

export type PlanName = keyof typeof PLAN_PRICES;

export function isPlanName(value: unknown): value is PlanName {
  return typeof value === "string" && value in PLAN_PRICES;
}

export function priceForPlan(input: {
  plan: PlanName;
  accountDiscountPercent: number;
  promoCode?: unknown;
}): number {
  const listPrice = PLAN_PRICES[input.plan];
  const afterAccountDiscount = listPrice * (1 - clampPercent(input.accountDiscountPercent) / 100);
  const afterPromo =
    afterAccountDiscount * (1 - clampPercent(promoDiscountPercent(input.promoCode)) / 100);
  return Math.round(afterPromo);
}

function clampPercent(value: number): number {
  if (!Number.isFinite(value) || value <= 0) return 0;
  return Math.min(value, 100);
}
```

No database, no request, no framework — it imports nothing. This is the file the finance team can be walked through, and the one the mobile app can share instead of reimplementing pricing a fourth time. `clampPercent` also means a corrupt `discount_percent` of `-500` can no longer produce a negative charge.

```
export async function changePlan(
  actor: Actor,
  input: ChangePlanInput,
  now: Date = new Date()
): Promise<SubscriptionView> {
  const subscription = await database.findSubscription(input.subscriptionId);
  if (!subscription) {
    throw new DomainError(404, "subscription_not_found", "Subscription not found");
  }
  if (subscription.user_id !== actor.id) {
    throw new DomainError(403, "not_your_subscription", "This subscription belongs to someone else");
  }

  const priceCents = priceForPlan({
    plan: input.newPlan,
    accountDiscountPercent: Number(subscription.discount_percent ?? 0),
    promoCode: input.promoCode,
  });

  const changedAt = now.toISOString();
  const updated = await database.transaction(async (tx) => {
    const row = await tx.applySubscriptionChange(input.subscriptionId, {
      plan: input.newPlan,
      priceCents,
      changedAt,
    });
    await tx.recordEvent({
      subscriptionId: input.subscriptionId,
      type: "plan_changed",
      detail: input.newPlan,
      actorId: actor.id,
      createdAt: changedAt,
    });
    await payments.updateSubscriptionPrice(String(subscription.payment_ref), priceCents);
    return toView(row);
  });

  mailer.enqueue(String(subscription.email), `Your plan is now ${input.newPlan}`);

  return updated;
}
```

The ownership check lives here, not in the handler and not in the UI, because this function is the only way to change a plan — there is no second door to forget to lock. `now` is injected so time is testable. The email is queued after the transaction commits, so a mail outage can no longer fail a payment that already succeeded.

```

export async function POST(req: Request) {
  const actorId = req.headers.get("x-user-id");
  if (!actorId) {
    return Response.json({ error: "Authentication required" }, { status: 401 });
  }

  try {
    const input = parseChangePlanInput(await req.json().catch(() => null));
    const subscription = await changePlan({ id: actorId }, input);
    return Response.json(subscription);
  } catch (error) {
    if (error instanceof DomainError) {
      return Response.json({ error: error.message, code: error.code }, { status: error.status });
    }
    throw error;
  }
}

```

Fourteen lines that know about HTTP and nothing about coffee subscriptions: establish who is calling, parse, delegate, map errors to status codes. A second caller — the mobile app, an admin tool, a scheduled job — reuses `changePlan` and inherits every rule for free.

### Step 3: what the tests show

Test Files 2 passed (2)  
Tests 29 passed (29)

Eight of them are the behaviour tests, run against both versions and passing identically — a customer using the feature correctly cannot tell the difference. That is what makes this a refactor rather than a rewrite.

The other tests are the ones that separate the two versions:

| DEFECT                                                | BEFORE                                                             | AFTER                                        |
|-------------------------------------------------------|--------------------------------------------------------------------|----------------------------------------------|
| Ben changes Ana's plan                                | <code>200</code> , write succeeds                                  | <code>403</code> , nothing written           |
| <code>'nope'</code> OR <code>'1'='1'</code> as the id | every subscription rewritten                                       | <code>404</code> , nothing matched           |
| Unknown plan <code>premium</code>                     | <code>NaN</code> written and sent to Stripe                        | <code>422</code> , nothing written           |
| Stripe times out mid-request                          | plan changed, unbilled, no audit row                               | rolled back completely                       |
| Mail provider fails                                   | request 500s, retry double-charges                                 | <code>200</code> , mail queued, charged once |
| Response body                                         | stale plan, <code>internal_notes</code> , <code>payment_ref</code> | current values, four public fields           |
| No auth header                                        | proceeds anyway                                                    | <code>401</code>                             |

## What I deliberately did not change

---

- **The discount stacking rule.** Still compounds. It may be wrong, but it now lives in one readable function so the conversation can happen. A refactor is not where you settle a pricing question.
- **Response field names.** `plan` and `id` keep their names even though the new object is camelCase elsewhere, because a mobile client I cannot redeploy reads them.
- **Rounding.** Still `Math.round` at the end. Same inputs, same pennies.
- **The endpoint, method and status codes for valid requests.**

Exactly two observable changes are not defect fixes: the response no longer includes internal columns, and the notification is queued rather than sent inline. Both go in the pull request description, because a reviewer should not have to discover a behaviour change by reading a diff.

## What this still does not fix

---

**A database transaction and an external API call are not atomic.** Wrapping the Stripe call in the transaction closes the common failure — we no longer commit a plan we failed to bill. The remaining window is the reverse: Stripe succeeds and our commit fails. Two things cover that, neither in this diff: an **idempotency key** derived from subscription, plan and price so a retry cannot double-charge, and the **nightly reconciliation job** from the [migration plan](#).

**Holding a transaction open across a network call holds a row lock.** Acceptable here — one row, a three-second payment timeout. If contention appears, the next step is an outbox: commit locally, publish an intent, let a worker call Stripe with retries.

**The test doubles are doubles.** `support/database.ts` is an in-memory Postgres stand-in with snapshot rollback. It makes this runnable in under a second with no setup; it does not prove Postgres behaves identically. In the real migration these same tests run against a real database in CI.

## What improved

---

The refactor removed a data breach, a mass-update vulnerability, a corrupt-price bug, silent billing drift, a double-charge on retry, and an information leak. The durable win is duller: **there is now one place where a plan change happens.** Every future rule — annual plans, regional pricing, pausing — has one home, one set of tests, and one door everything comes through.